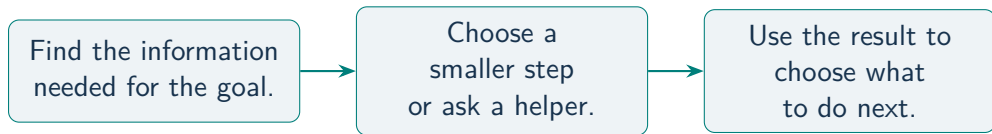


Can a model learn useful steps for an unfamiliar task?

A recursive language model (RLM) can use code and ask another model to handle a smaller task. A helper can, in turn, ask its own helper.



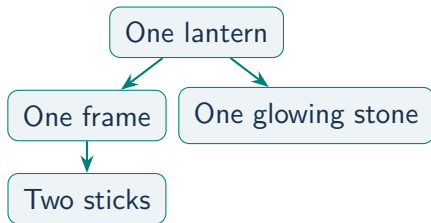
Today's focus: Can we improve how a small model learns these steps by improving its worked examples?

Our longer-term aim is useful task decomposition. Today's strongest evidence concerns learning tool actions, not general recursive reasoning.

A crafting game makes the outcome easy to check.

In TextCraft-Synth, success means making the requested number of new items. The game checks the inventory increase when the model finishes.

Goal: make one lantern.



Arrows mean “needs.”

Simplified illustration. Real tests use unfamiliar names and longer chains. Scores count whole-task success, with no partial credit.

Start with two sticks and one glowing stone.

1. Look up the lantern and frame recipes.
2. Make the frame from the sticks.
3. Combine the frame and stone into a lantern.

Show the relevant fact before the action.

Supervised fine-tuning (SFT) teaches a next action from an example's goal and observations.

Original input

Goal: make a lantern.

History: no frame mentioned.



Next action to learn

Look up the frame recipe.

Repaired input

Goal: make a lantern.

Lookup: a lantern needs a frame.



The same action to learn

Look up the frame recipe.

We rebuild the earlier lookup history. **Each example keeps the same action to learn.**

Simplified illustration. The repair uses a known solution; input histories, their lengths and computing work change.

The repair helped after retraining and on more goals.

Each bar counts successful crafting attempts out of 32. Both models learned the same actions for the same number of training updates.



Each row tests eight goals in two recipe worlds, twice each. There are **16 distinct goals in total**; the first two rows reuse the same goals.

Selected TextCraft-Synth panels, not the full benchmark; Qwen3-4B. More training helps the original examples too. A teacher that discovers recipes in order also performs well.

Let code write the known ingredient list.

After a recipe lookup, code can write the ingredients for the **model's chosen item and quantity**.

Example: for “make one lantern,” code writes “one frame and one glowing stone.”

Model writes the full action.

42%
of attempts succeeded.

Code writes the ingredients.

50%
of attempts succeeded.

We changed the interface, not the model's weights. Code supplies no materials.

Selected TextCraft-Synth panels: 323/768 versus 381/768 whole-task successes. There are 48 distinct goals, four recipe worlds and repeated seeds, not 768 independent problems.

What did we learn from reward training?

Reinforcement learning (RL) rewards attempts that actually finish the task. We started with an SFT-trained model and trained on eight crafting goals.



Every point tests the **same eight other goals twice each**. The extra success after two updates did not persist.

Lesson: A gain must survive new examples and comparison with simply continuing SFT.

Selected TextCraft-Synth training-split goals; recipes overlap. No checkpoint was picked for its score here. Three small updates are not a verdict on RL.

Other datasets helped us ask what actually improved.

MuSiQue and HotpotQA ask questions about several documents. Our trained question-splitting system did not clearly beat answering directly, and used much more text.

AG News and DBpedia ask for a topic label. Small RL gains on news weakened on fresh examples and did not clearly transfer to encyclopedia entries.

MRCR asks for a particular earlier reply in a conversation. RL mostly improved exact reproduction of the answer, not finding the right reply.

These are different studies, not a shared leaderboard. A higher score does not by itself show better decomposition. The reading guide explains each test and its limits.

Can the teaching repair survive stronger tests?

Research question

Can rebuilding what the learner sees make existing worked examples more useful, without changing the actions we teach?

Test new problems. Use new recipe structures, not just more attempts at familiar goals.

Use stronger comparisons. Include a teacher that discovers recipes in order, and give the original examples more training.

Find the limits. Try another model and test whether the gain is mainly easier copying of item names.

A paper candidate, not an established general method. What evidence would make it useful beyond this game?